

SafeTracker — Technical Documentation

Consent-First Real-Time GPS Tracking Ecosystem

This document provides a comprehensive technical overview of the **SafeTracker** location mapping system located at **E:/PROJECTS/INTERN/locaton_tracking**. It details the ethical tracking frameworks, React Native app, Node.js backend, and map interfaces.

1. Executive Summary

SafeTracker is an ethical, consent-first location-tracking application. It features a React Native client app for transmitting background coordinates, an Express.js server hosted on Render.com, and a real-time monitor web dashboard utilizing Leaflet.js to plot locations, accuracies, and paths.

2. Privacy & Ethical Guidelines

This platform is engineered strictly for **consensual** tracking (e.g. family location sharing, logistics). It enforces privacy via:

- **Explicit Opt-in:** The app requires the user to read and check a consent screen.
 - **Persistent Notification Bar:** A permanent Android notification **■ SafeTracker Active** is displayed whenever tracking is active.
 - **Granular Stop Switch:** Users can deactivate tracking directly within the mobile client at any time, instantly halting background service tasks.
 - **Device ID Visibility:** The hardware device ID is displayed inside the app header for total visibility.
-

3. Technology Stack & Directory Structure

```
locaton_tracking/ ■■■ server/ ← Express.js + WebSocket server ■■■ dashboard/ ←  
Leaflet.js web monitoring client (HTML/JS) ■■■ SafeTracker/ ← React Native Mobile App  
(Expo SDK 54)
```

- **Mobile Client:** React Native, Expo SDK 54, **expo-location**, **expo-task-manager** (handles OS background location threads), and AsyncStorage (offline database buffering).
 - **Backend Cloud:** Node.js, Express, **ws** (WebSockets).
 - **Web Dashboard:** Vanilla HTML5/JS, Leaflet.js (map rendering), CartoDB dark-themed tile vectors.
-

4. Technical Features

A. Offline Storage & Auto-Sync

If internet drops:

1. Coordinates are recorded by the background task.
2. Coordinates are stored as JSON arrays in **AsyncStorage** (buffers up to 2000 coordinates, ~5.5 hours of tracking).
3. The app displays an amber status banner: **X locations queued offline**.
4. Once internet returns, NetInfo triggers a **POST** request to **/api/location/batch** to upload coordinates in a single optimized payload.

B. Hardware Device ID Resolution

To support multi-device dashboards, each device resolves a unique ID:

1. **androidId** (Android) or **identifierForVendor** (iOS) is pulled.
2. Stored in AsyncStorage and cached in memory.
3. Multiple phones appear as separate cards on the central map.

5. Endpoints & Run Guide

API Endpoints

- **POST /api/location**: Records a single coordinate.
- **POST /api/location/batch**: Ingests queued offline coordinates.
- **GET /api/devices**: Lists all active unique device IDs.
- **GET /api/location/:deviceId/history**: Retrieves routing paths for a target device.

How to Start Server

```
cd E:/PROJECTS/INTERN/locaton_tracking/server npm install node server.js
```

Server runs on port 4000.

How to Start Expo App

```
cd E:/PROJECTS/INTERN/locaton_tracking/SafeTracker npm install npx expo start
```